Pattern:

INDEX

ANITS

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
①	1/7	Design static webpage using HTML components	CO1	15	
②	8/7	Design webpage using HTML5 & CSS3	CO1	14	
③	15/7	Write a program using Javascript (arrays, functions)	CO1	15	
④	22/7	Develop Dynamic webpage using Javascript	CO1	15	
⑤	29/7	Implement modules in NodeJS	CO2	14	
⑥	5/8	Develop mini application using Express and NodeJS	CO2	14	
⑦	12/8	create a Functional component and class component using React	CO3	14	
⑧	16/9	Implement http methods using JSON server	CO3	14	
⑨	23/9	Develop CRUD operations using MongoDB	CO4	18	
⑩	30/9	Develop Main project using Express, React and MongoDB / MY SQL.	CO4	15	

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech – ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026	Semester:	III 2nd year, 1st Sem.
Name of the Student:	A Hemanth	Roll Number:	A24126552277
Name of the Lab:	FSPP Full stack	Timings:	9:40 to 12:40

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva- Voce (2Marks)			
02/7	01	4	4	5	2	15	10	Pr
08/7	02	4	4	5	1	14	10	Pr
15/7	03	4	4	5	2	15	10	Pr
22/7	04	4	4	5	2	15	10	Pr
29/7	05	3	4	5	2	14	10	Pr
05/8	06	4	4	5	2	15	10	Pr
12/8	07	4	4	5	1	14	10	Pr
16/9	08	3	4	5	2	14	10	Pr
23/9	09	4	4	5	2	15	10	Pr
30/9	10	4	4	5	2	15	10	Pr

Name of the Faculty: Prasanna Priya

Faculty Signature: Pr

Experiment - 01

Introduction to HTML and CSS

* HTML (Hyper text markup Languages) is the standard markup Languages used to create and structure content on web pages. It defines the basic structure of a webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a web page.

→ CSS (Cascading style sheets) is a style sheet Languages used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, font, backgrounds, spacing, borders, alignment, dimensions and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen size and devices. CSS can be applied using inline, internal, or external stylesheets.

→ HTML and CSS work together to develop webpages where HTML provides the structure and content while CSS provides the styling, layout, visual presentation. The combination of both forms the foundation of modern web development and enables the creation of organized, attractive and user-friendly web interfaces.

Difference between MERR stack and Full stack:-

* MERN stack is a Javascript-based technology stack used for developing modern web applications. MERN stands for MongoDB, Express.js, React.js, Node.js. MongoDB is used for database management, Express.js handles the backend framework and API's, React.js is used to develop frontend user interface and Node.js provides the server-side runtime environment.

→ Full-stack development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic, managing databases.

webpages
content
isual
forms
lopment
attractive

stack:-
chnology
applica
ress is,
for
es the
is
e and
le
elopment
web
.
ologies
lementary

HTML Components

① HTML Tags: Define the structure and elements of a webpage.

② Heading:- used to display titles and headings using <h1> to <h6>

③ Paragraphs: used to display text using the <p> tag

④ Links: used to navigate between webpages using <a> tag

⑤ Images: used to display images using <image> tag.

⑥ Lists: used to create ordered and unordered lists using <ol>, <ul>,

⑦ Tables: used to organize data in rows and columns

⑧ Forms: used to collect user input using elements such as text boxes, buttons and checkboxes

⑨ Div and span: used to group and organize webpage content.

⑩ Semantic Elements: Elements such as <header>, <nav>, <selection>, <article>, <footer>, provide meaningful webpage structure

CSS Components:

1. selectors - select HTML elements to apply style
2. properties - Define the visual characteristic of elements.
3. values - specify the setting of a css property.
4. Colors - Define text, background, border colors
5. Fonts - control font family, size, style and weight.
6. Backgrounds - set background colors and images
7. Border - Define the border style, width and radius.
8. Margin - controls the space outside an element.
9. Padding - controls the space inside element.
10. layouts - controls the arrangement of elements using flexbox, Grid and Positioning.

HTML VS HTML5

HTML	HTML 5
*Older version of HTML	* Latest major version of HTML
* Limited multimedia support	* supports audio and videos
* uses external plugins for multimedia	* No external plugins needed
* Has fewer semantic tags	* provides semantic tags
* No <canvas> element	* supports <canvas>
* Limited web storage	* supports local storage
* Complex DOCTYPE Declaration	* simple <!DOCTYPE HTML>

CSS VS CSS3

CSS	CSS3
*Earlier version of CSS	* Advanced version of CSS
* Basic styling features	* Provides advanced styling features
* Limited animations	* supports animations and transitions
* No flexible layout systems	* supports flexbox and grid
* Limited visual effects	* supports shadows, gradients effects.
* No media queries	* supports media queries
* uses a single specification	* Divided into different modules.

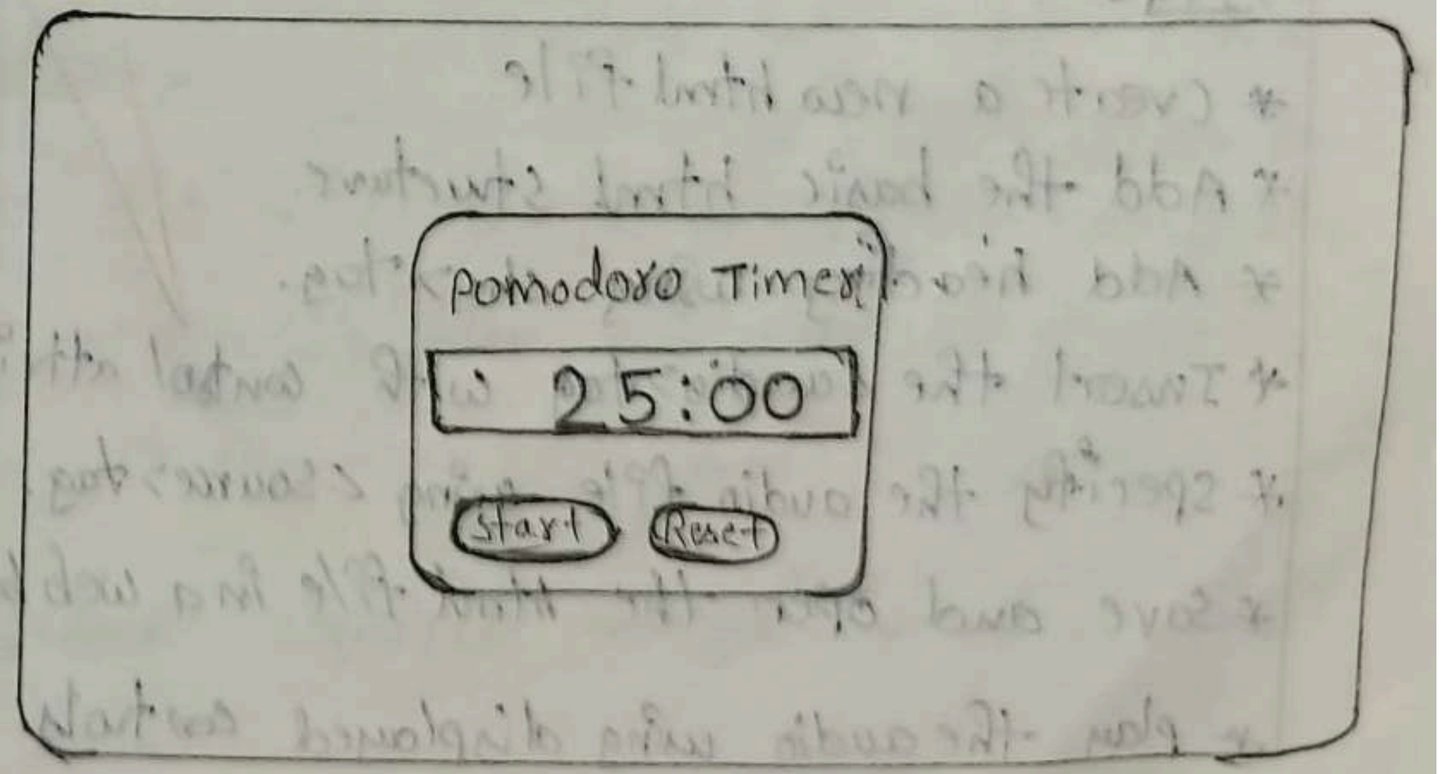
Experiment-02

Design a static webpage using HTML & CSS.

Pomodoro Timer using HTML, CSS.

```
<!DOCTYPE html>
<html>
<head>
  <title> pomodoro Timer </title>
  <style>
     body {
        font-family : Arial;
        background : #f5f5f5;
        text-align : center;
        padding    : 50px;
     }
  timer {
        width : 300px;
        margin : auto;
        padding : 30 px;
        background : white;
        border-radius : 15px;
        box-shadow : 0 0 10px gray;
  }
  h1 {
    color : #e74c3c;
  }
  time {
    font-size : 50 px;
    margin : 20px;
  }
    button {
        padding : 10px 20px;
        margin : 5 px;
        border : none;
        border-radius : 5px;
        background : #e74c3c;
        color : white;
    }
  </style>
</head>
<body>
  <div class = "Timer">
    <h1> Pomodoro Timer </h1>
    <div class = "time"> 25:00 </div>
      <button> start </button>
      <button> Reset </button>
  </div>
</body>
</html>
```

output:



i) Design an HTML code using audio tags.

code:

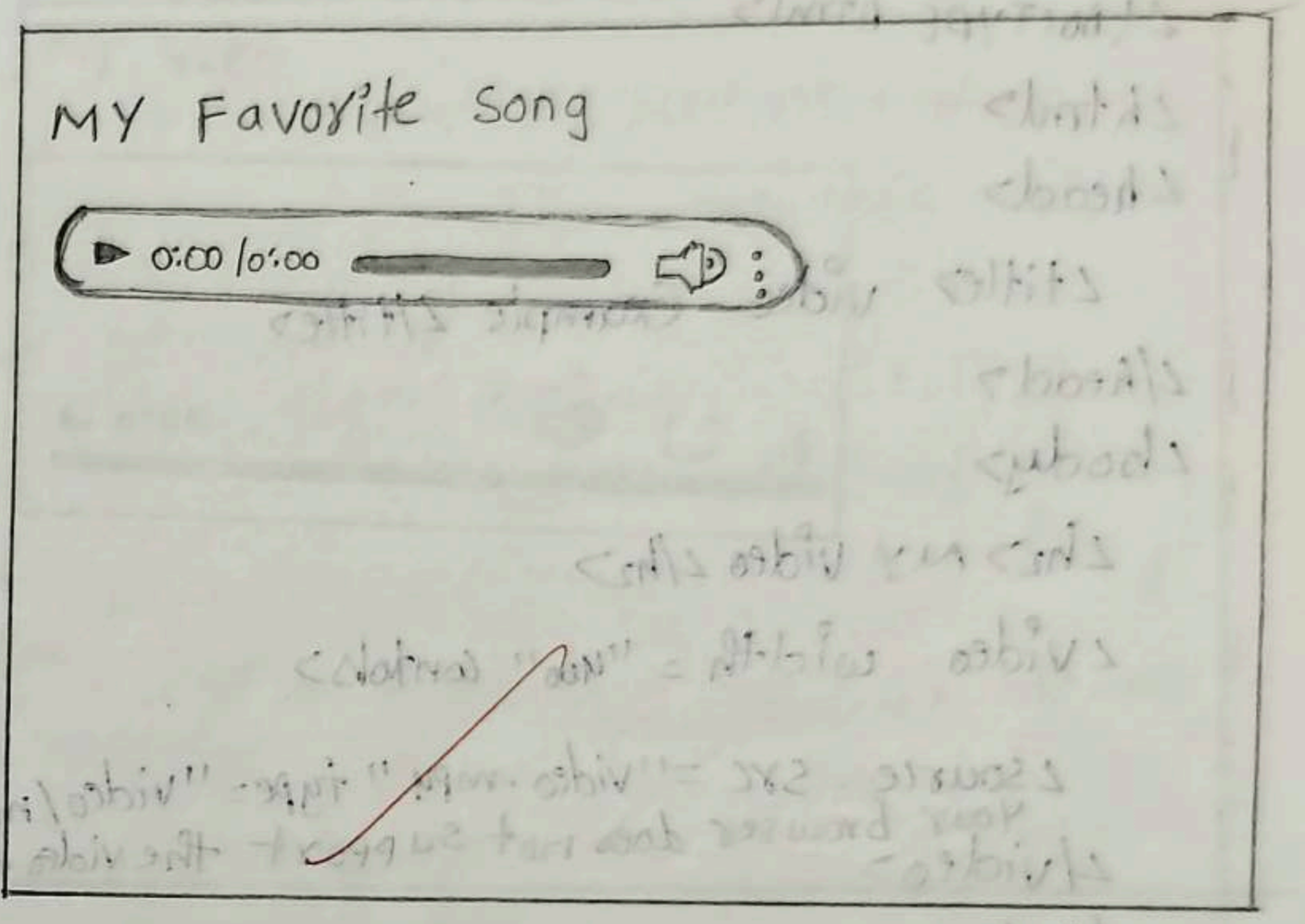
```
<!DOCTYPE html>
<html>
<head>
  <title> Audio Example </title>
</head>
<body>
  <h2> My Favourite song </h2>
  <audio controls>
    <source src="song.mp3" type="audio/mpeg">
    Your browser does not support the audio tag
  </audio>
</body>
</html>
```

o/p: Displays an audio player with play, pause and volume controls.

Steps:

* Create a new html file.
* Add the basic html structure.
* Add heading using <h2> tag.
* Insert the <audio> tag with control attributes
* specify the audio file using <source> tag.
* Save and open the html file in a web browser.
* play the audio using displayed controls.

output:



ii) Design a HTML code using video tags.

Code:

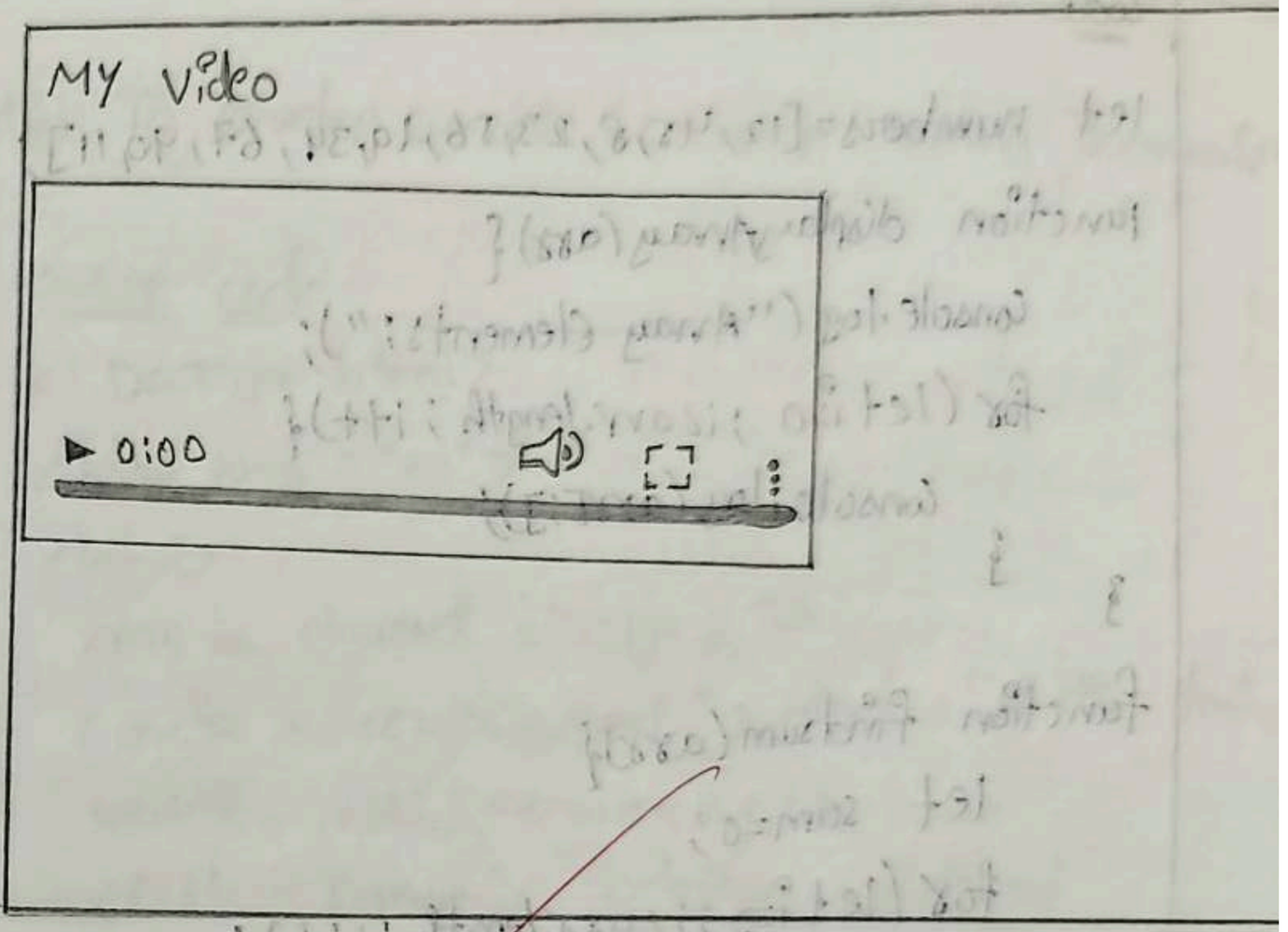
```
<!DOCTYPE html>
<html>
<head>
  <title> Video Example </title>
</head>
<body>
  <h2> MY Video </h2>
  <video width = "400" controls>
    <source src = "video.mpy" type = "video/mpy">
    Your browser does not support the video tag
  </video>
</body>
</html>
```

o/p: Displays a video player with play, pause, volume, and full screen controls.

Steps:

*Create a new HTML file.
*Add a basic html structure
* Add a heading using the <h2> tag.
* Insert the <video> tag with controls attribute
* specify the video file using <source> tag.
* save the file and open it in web browser
*play the video using displayed controls

Output:



write

Experiment-03

3) Write a program using Javascript (arrays, functions)

code:

```
let numbers=[12,45,8,23,56,19,34,67,90,11];
Function displayArray(arr){
    Console.log("Array Elements:");
    for(let i=0 ;i<arr.length; i++){
        Console.log(arr[i]);
    }
}
function findsum(arr){
    let sum=0;
    for(let i=0;i<arr.length;i++){
        sum+=arr[i];
    }
    return sum;
}
displayArray(numbers);
Console.log("sum=", find sum(numbers));
```

Output: Array Elements: 12
45
8
23
56
19
34
67
90
11
sum=365

Experiment-04

4) Develop Dynamic webpage using Javascrip

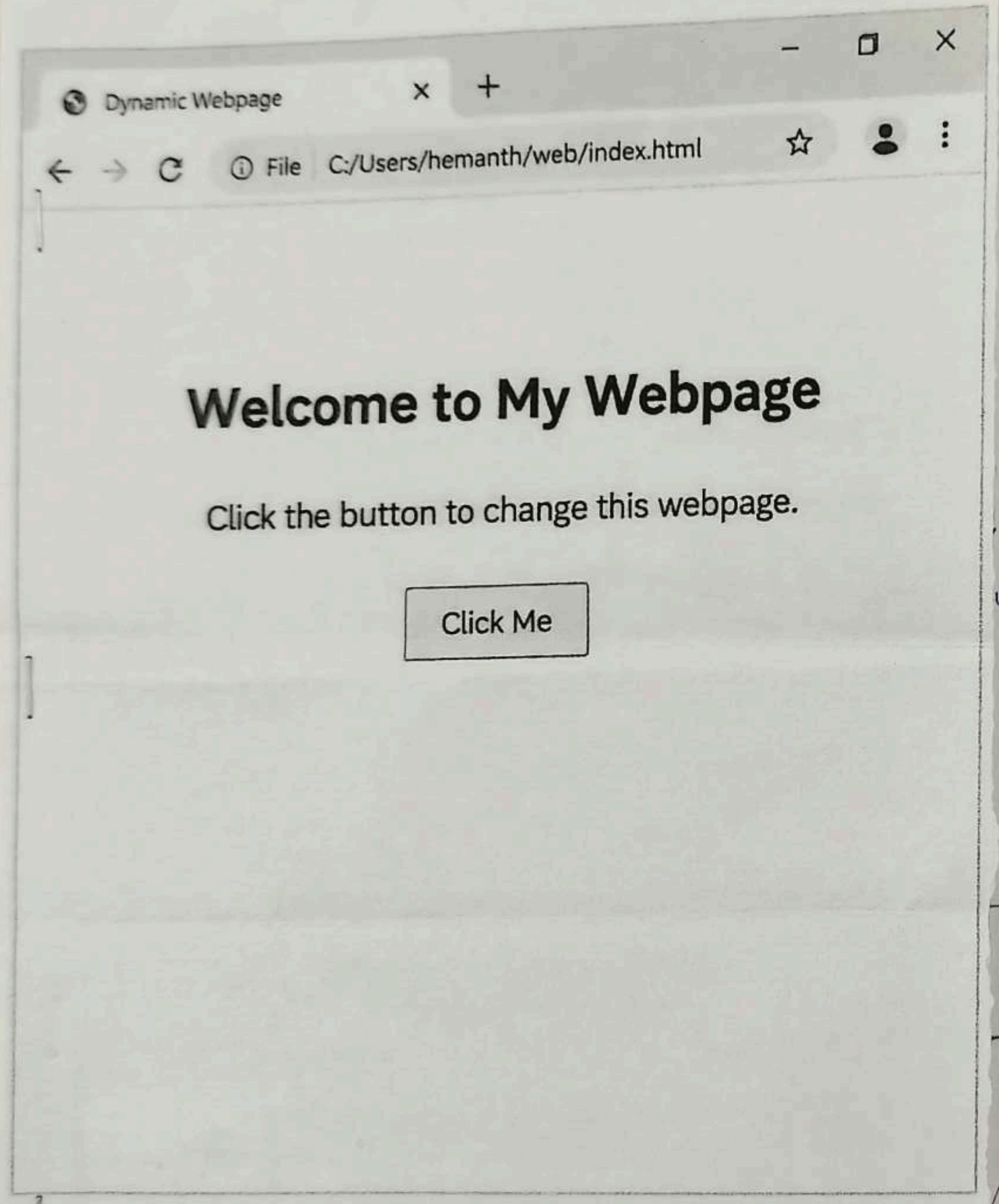
Aim: To Develop Dynamic webpage using Javascrip

source code:

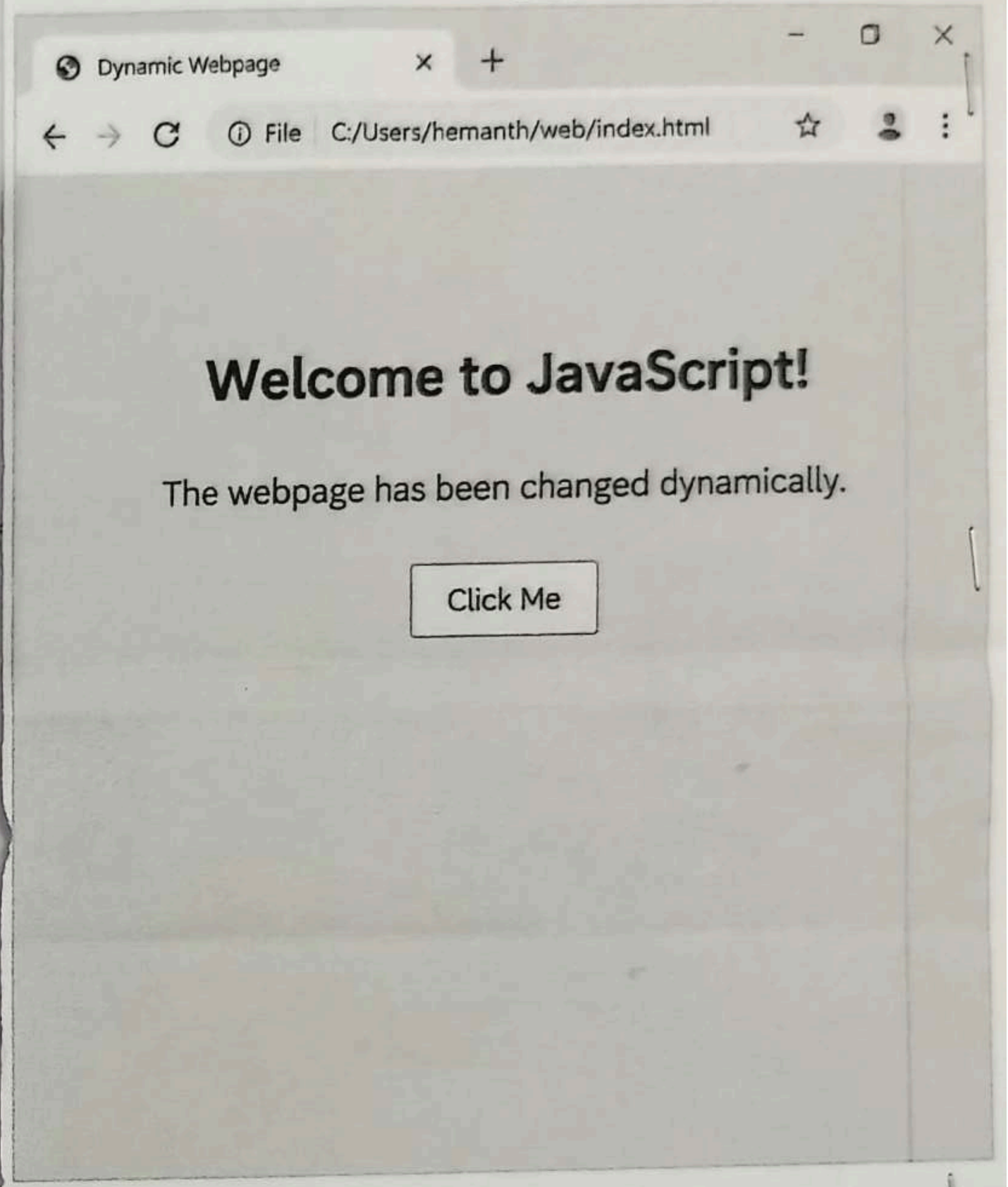
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device
  width, initial-scale=1.0">
  <title> Dynamic webpage </title>
<style>
 body {
    font-family: Arial, sans-serif;
    text-align: center;
    margin:0;
    background-color: #f2f2f2;
 }
header{
    background-color: #333;
    color: white;
    Padding: 20px;
}
• Container {
  margin: 40px auto;
  Padding: 25px;
```

```
background : white;
width : 80%;
max-width : 400px;
border-radius : 10 px;
box-shadow: 0 0 10px gray;
}
input{
    padding : 12px;
    width: 85%;
    margin : 10px 0;
    border : 1px solid #ccc;
    border-radius : 5 px;
}
button{
    padding : 10 px 20px;
    margin : 10px 5px;
    border : none;
    border-radius : 5px;
    background-color : #333;
    color : white;
    cursor : pointer;
}
button:hover {
    background-color : #555;
}
#message{
    margin-top : 20px;
    font-size : 18 px;
    font-weight : bold;
}
</style>
</head>
<body>
<header>
    <h1> Dynamic login page </h1>
    <p> Javascript Lab Demo </p>
</header>
<div class ="container">
    <h2> Login </h2>
<input type = "email" id="email" placeholder = "Enter Your email">
<input type="password" id = "password" placeholder = "
Enter Your password" >
<br>
<button onclick ="Login()" > submit </button>
<button onclick ="resetform()" >Reset </button>
<div id = "message"></div>
</div>
<script>
    function login() {
    let email = document.getElementById("email").value;
let password = document.getElementById("password").value;
let message = document.getElementById("message");
```

```
if(email === "" || password === "") {
    message.innerHTML = "please enter both email
                         and password.";
    message.style.color = "red";
```



Before clicking the button



After clicking the button

```
submit.addEventListener("click", function(event) {
    event.preventDefault();
    if (email === "admin@example.com" && password === "password123") {
        message.innerHTML = "Login successful! Welcome, admin.";
        message.style.color = "green";
    } else {
        message.innerHTML = "Invalid email or password.";
        message.style.color = "red";
    }
});
```

Experiment-05

5) Implement modules in Node JS

Aim: To Implement modules in Node JS

source code:

modules :- Project/
- add.js
- sub.js
- mul.js
- div.js
- app.js

→ must be same folder

① add.js:

```
function add(a,b){
    return a+b;
}
module.exports=add;
```

② sub.js:

```
function sub(a,b){
    return a-b;
}
module.exports=sub;
```

③ mul.js:

```
function mul(a,b){
    return a*b;
}
module.exports=mul;
```

④ div.js

```
function div(a,b){
    return a/b;
}
module.exports=div;
```

⑤ app.js :-

```
const add = require("./add");
const sub = require("./sub");
const mul = require("./mul");
const div = require("./div");
let a=20;
let b=10;
console.log("Addition:", add(a,b));
Console.log("substraction:", sub(a,b));
Console.log("multiplication:", mul(a,b));
Console.log("Division:", div(a,b));
```

Bash: node app.js

output:

```
Addition : 30
substraction : 10
multiplication: 200
Division : 2
```

Experiment 06

6, Develop mini application using express and NodeJS

Aim: To develop mini application using Express and Node JS

source code & process:

Install node.js on your computer, after installation, check it with:

Bash

```
node -v
npm -v
```

→ **Install Express**

```
npm install Express
```

structure

```
Express-mini-app/
  ├ Express.js
  └ Javascript/
      └ Index.html
```

* The folder name Javascript must match the name used in:

JS

```
app.use(express.static('Javascript'));
```

Express.JS

```
const express = require('express');
const app = express();
const port = 3000;
// middleware to read JSON data
app.use(express.json());
// serve frontend files from Javascript folder
app.use(express.static('Javascript'));
// Login API
app.post('/login', (req, res) => {
  const { email, password } = req.body;
  if (email === 'admin@gmail.com' && password === '12345') {
    res.json({
      success: true,
      message: 'login succesful'
    });
  } else {
    res.json({
      success: false,
      message: 'Invalid Email or Password!'
    });
  }
});
// about route
app.get('/about', (req, res) => {
```

```
});

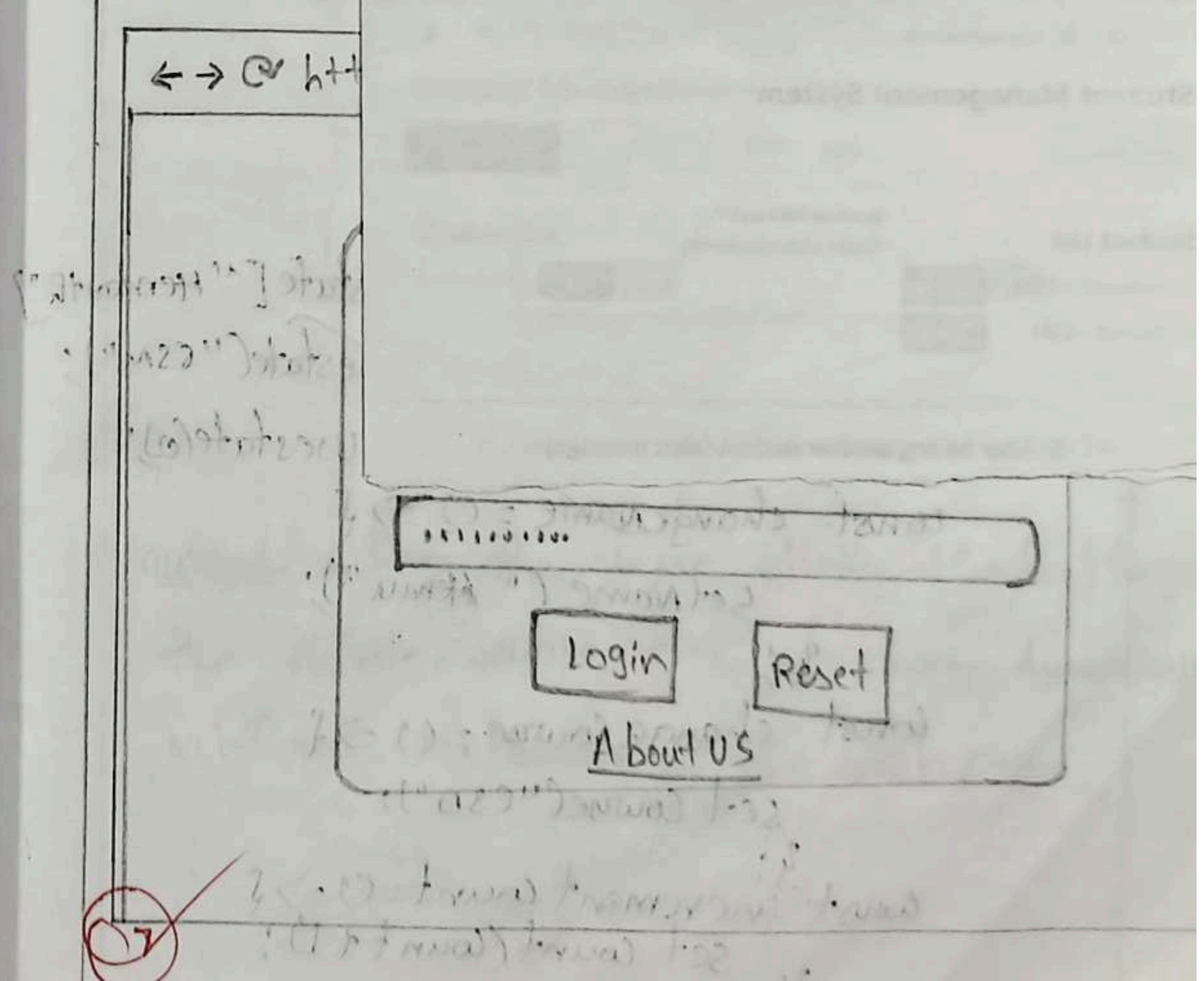
// start server
app.listen(port, () => {
  console.log('server is running on port 3000');
  console.log('visit http://localhost:${port}');
```

Javascript/index.html

```
<!DOCTYPE html>
<html>
<head>
  <title> Express 'Mini' Application </title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
      margin-top: 80px;
      background-color: #f2f2f2;
    }
    .container {
      background: white;
      width: 350px;
      margin: auto;
      padding: 30px;
      border-radius: 10px;
      box-shadow: 0 0 10px gray;
    }
    input {
      width: 90%;
      padding: 10px;
      margin: 10 px;
    }
    button {
      padding: 10 px 20 px;
      margin: 10 px;
      cursor: pointer;
    }
    #result {
      margin-top: 20 px;
      font-weight: bold;
    }
  </style>
</head>
<body>
  <div class = "container">
    <h2> Login </h2>
    <input type= "email" id="email" placeholder = "enter email">
    <input type="password" id = "password" placeholder= "enter password">
    <br>
    <button onclick="login()" >Login </button>
    <button onclick = "resetForm()">Reset </button>
    <p id = "result"></p>
    <a href = "/about" >about us </a>
  </div>
```

```
</script>
function login(){
  const email=document.getElementById("Email").value;
const password=document.getElementById("password").value;
fetch('/login',{
   method:'POST'
   headers:{
      'Content-TYPE':'application/json'
},
body:JSON.stringify({
   email:email,
   password:password
})
})
.then(response=>response.json())
.then(data=>{
   document.getElementById("result").innerText=
                                   data.message;
});
}
function resetform(){
document.getElementById("email").value="";
document.getElementById("password").value="";
document.getElementById("Result").value="";
```

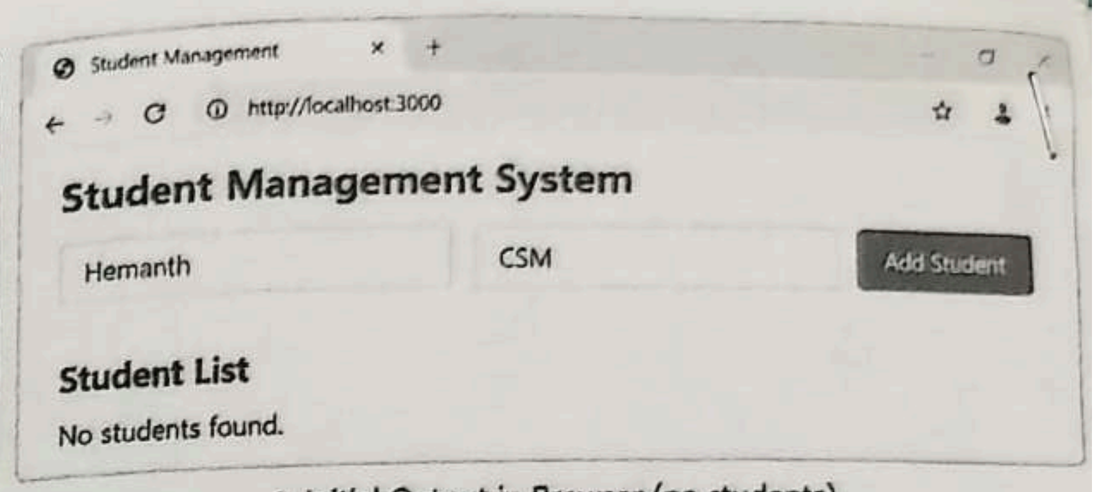
```
</script>
</body>
</html>
Commands to
  cd exp
Install expr
  npm ins
start the s
  node
You should
server is
visit h
```



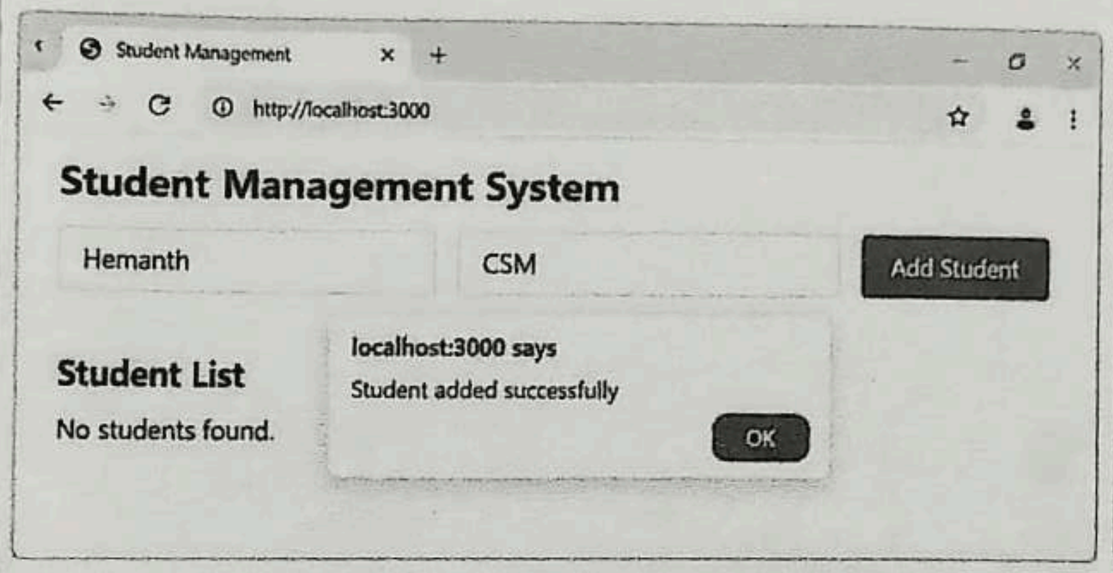
```
<script>
function login(){
  const email=document.getEl
const password=document.getEl
  fetch('/login',{
    method:'POST',
    headers:{
      'Content-Type':'app
},
    body:JSON.stringify({
      email:email,
      password:password
    })
})
.then(response=>respons
.then(data=>{
    document.getElementById
  });
}
function resetform(){
document.getElementById("email").value="";
document.getElementById("password").value="";
document.getElementById("Result").value="";
}
```



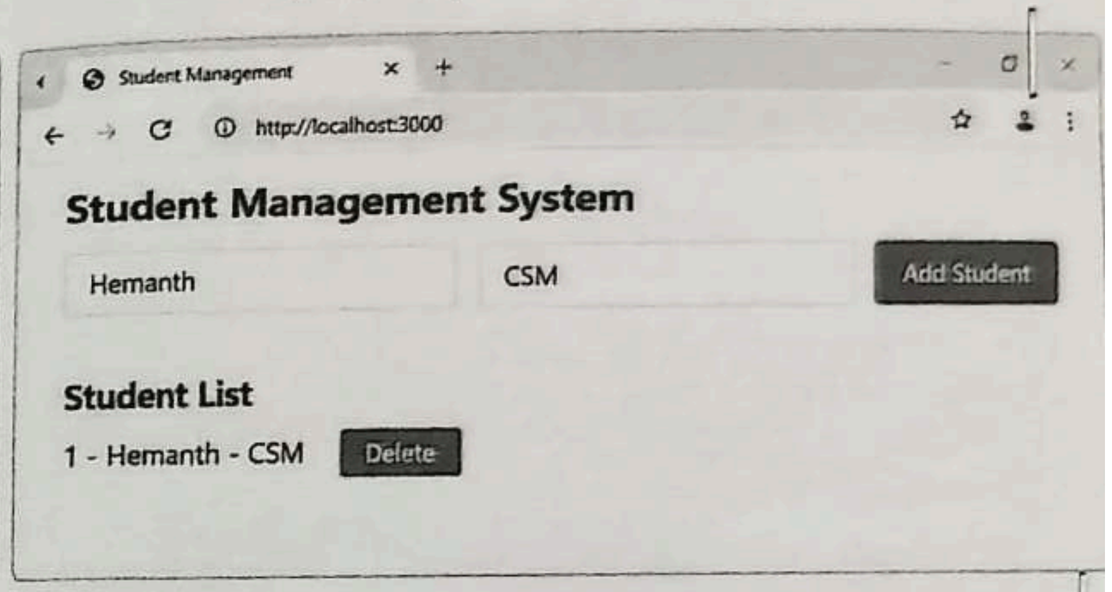
1. Terminal Output (after running `node server.js`)



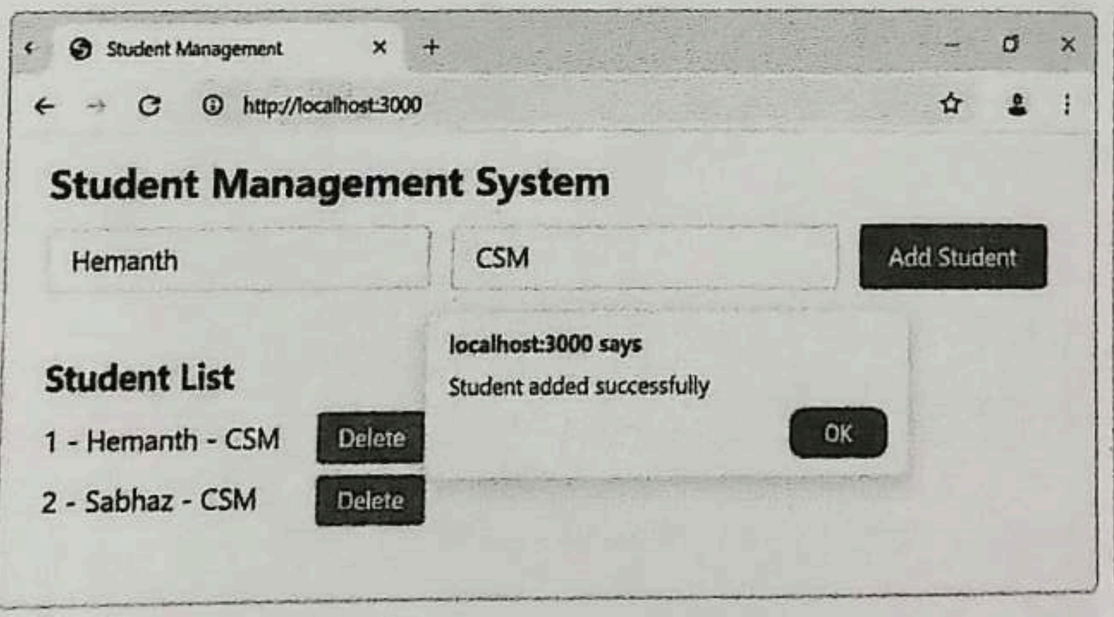
2. Initial Output in Browser (no students)



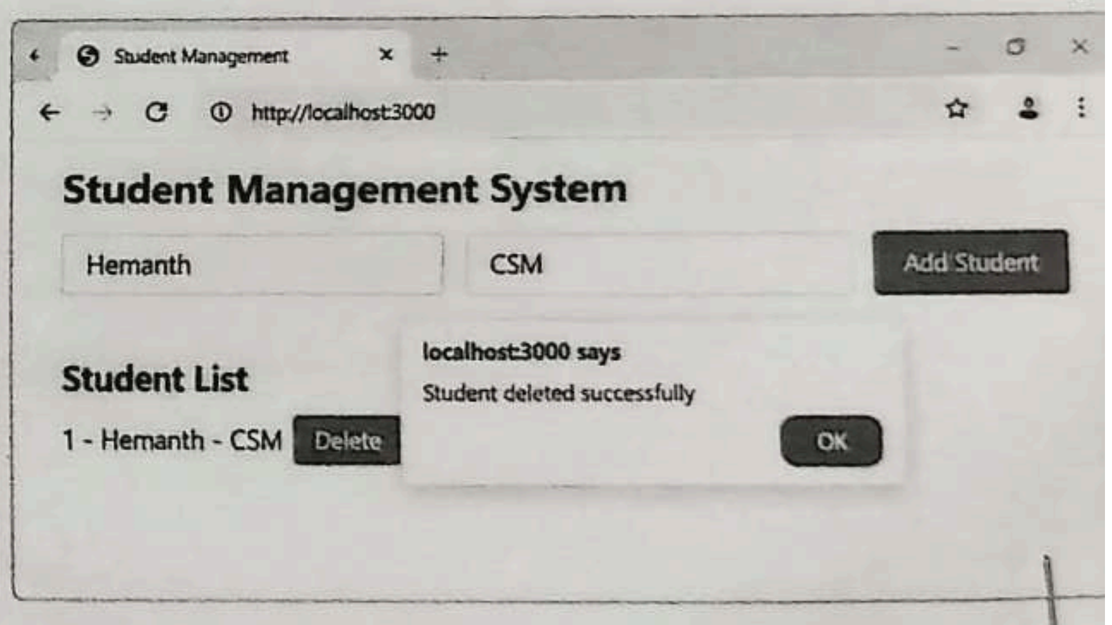
3. After adding a student (alert message)



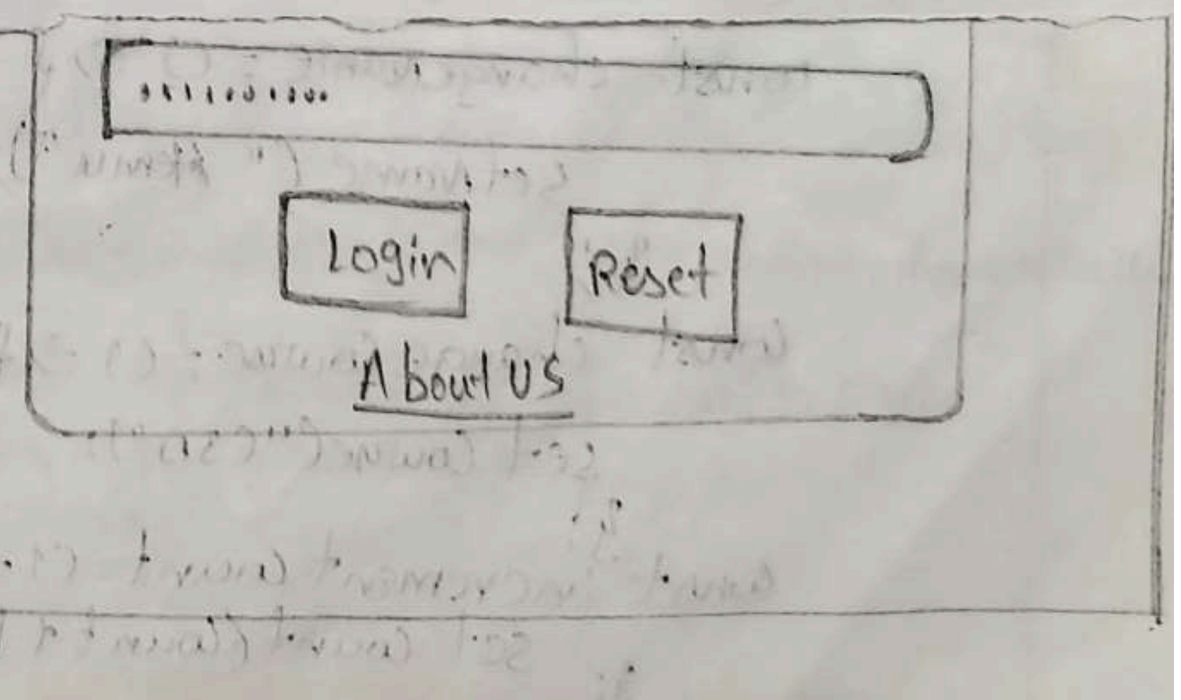
4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)



Experiment-07

7. Create an Functional Component and class component using React.

Aim: To create an Functional Component and class component using React

Source code:

Functional Component:

A functional component is a Javascript function that returns JSX to display the user interface. It uses React hooks such as useState to manage state and update the webpage dynamically.

program:

```
import React, {useState} from
'react':
function studentFunctional() {
    const [name, setName] = Usestate ["Hemanth"]
    const [Course, setCourse] = Usestate("BSM");
    const [count, setCount] = Usestate(0);
    const changeName = () => {
        setName ("Hemu");
    };
    const changeCourse = () => {
        setCourse("CSD");
    };
    const incrementCount = () => {
        setCount(count+1);
    };
    return (
    <div style={{
        border: "2px solid blue",
        padding: "20px",
        margin: "20px"
        borderRadius: "10px",
        textAlign: "center"
        }}>
    <h1> StudentDetails </h1>
    <h2> Functional Component </h2>
    <p><b> name: </b>{name}</p>
    <p><b>Course: </b>{Course} </p>
    <p><b>Button clicks: </b>{count} </p>
    <button onClick = {changeName}>
        changeName </button>
    <button onClick = {changeCourse} change
        Course </button>
    <button onClick = {incrementCount}>
        increment </button>
    </div>
    );
```

output: when the user clicks the button, the name, course & count change dynamically without reloading the webpage.

Class component

A class component is an ES6 Javascript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState().

Program:

```
import React, {Component} from
'react';
class studentclass extends Component{
    constructor(props){
        super(props);
        this.state={
            name: "Hemanth";
            course: Artificial Intelligence";
            count: 0
        };
    }
    changeName=()=>{
        this.setState({
            name:"Hemu"
        });
    };
    changeCourse=()=>{
        this.setState({
            course:"CSM"
        });
    };
    incrementCount=()=>{
        this.setState((prevState)=>
            count=prevstate.count+1
        });
    };
render(){
    return(
        <div style={{
            border:"2px solid green";
            padding:"20px";
            margin:"20px";
            borderRadius:"10px";
            textAlign:"center";
        }}>
        <h1> Student Details</h1>
        <h2> Class Component</h2>
    <p><b> Name: </b>{this.state.name
    <p><b>Course: </b>{this.state.cour
    <p><b>Button clicks: </b>{this.state
    <button onclick={this.changeN
        change Name </
    <button onclick={this.change cour
        change course </bu
    <button onclick={this.increment coun
        Increment </button
    </div>
```

React Components

Student Details
Functional Component

Name: Hemanth
Course: CSM
Marks: 100

Change Name | Change Course | Increment

Student Details
Class Component

Name: Hemanth
Course: CSM
Marks: 100

Change Name | Change Course | Increment

```
name : "Hemu"
});
};
changeCourse2 = () => {
  this.setState({
    course : "CSM"
  });
};
```

```
Increment </button>
</div>
```

Experiment-8

8) Implement http methods using JSON server.

Aim: To Implement http methods using JSON server.

Theory) Json server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operation without developing a backend server from search. HTTP methods are used to communicate b/w a client and a server.

* GET - retrieves data from the server.
* POST - Adds new data to the server.
* PUT - Replaces an existing record.
* PATCH - updates selected fields of a record.
* DELETE - removes a record from the server.

a) create a JSON server:

step-1:- Initialize the project

open the vs code terminal & execute the following commands:

```
mkdir JSON server
cd JSON server
npm init -y
npm install json server
```

step-2: create db.json

create a file named db.json and add the following student data.

```
{
 "students": {
   {"id": "1",
   "name": "Hemanth",
   "Course": "CSM",
   "marks": 90;
   };
  {"id": "2",
   "name": "Hemuu",
   "Course": "CSM",
   "Marks": 90;
   }
  {
   "id": "3";
   "name": "Hemanthu";
   "Course": "CSE";
   "Marks": 88
  }
```

step-3: start the JSON server

for the current JSON server version, execute

```
npx json.server db.json
```

The server start at:

http://localhost:3000

The student records can be account using

http://localhost:3000/students

2. Implementation of HTTP methods:

a, GET Method: The GET method is used to retrieve student records from JSON server.

code:

```
fetch('http://localhost:3000/students')
*then (response => response.json())
*then (data => {
        console.log(data);
    })
* catch (error => {
    console.log(error);
  })
```

b, POST Method: The POST Method is used to insert a new student record into the server.

code: fetch (http://localhost:3000/students:{

```
    method:'post')
    headers: {
     'Content-Type': "application/json"
     },
    body: JSON.stringify({
        name: 'Rahul',
        course: 'cse',
        marks: 92
    })
  })
• then.(response => response.json())
• then (data => {
        console.log(data);
  })
• catch (error => {
        console.log(error);
  });
```

OP:

4 PUT Method: The PUT method is used to replace an existing student record with updated data.

Code:
```
fetch('http://localhost:3000/students/1',{
    method:'PUT',
    headers:{
    'Content-Type':'application/json'
    },
    body:JSON.stringify({
        name:'Hemanth',
        course:'AIML'
        marks:95
    })
})
.then(response=>response.json())
.then(data=>{
    console.log(data);
})
.catch(error=>{
    console.log(error);
});
```

O/P:

D, PATCH Method: The PATCH method is used to update only selected fetch of an existing student record.

Code:
```
fetch('http://localhost:3000/students/2',{
    method:'PATCH',
    headers:{
    'Content-Type':'application/json'
    },
    body:JSON.stringify({
    marks:96
    })
})
.then(response=>response.json())
.then(data=>{
    console.log(data);
})
.catch(error=>{
    console.log(error);
});
```

O/P:

6) DELETE Method: The DELETE method is used to remove an existing student record from the server.

Code:
```
fetch('http://localhost:3000/students/3',{
    method:'DELETE'
    })
    then(response =>{
        if(response.ok){
        console.log("student deleted succesfully");
        }
    })
    .catch (error=>{
        console.log(error)
    });
```

O/P:

*After executing the POST, PUT, PATCH & DELETE operations in sequence, the remaining student will be;

O/P:

Thus, HTTP methods. GET, POST, PUT, PATCH and DELETE were successfully implemented using JSON server. The CRUD operations were performed on student records. The data was retrieved, inserted, updated & deleted using

6) DELETE Method: The DELETE method is used to remove an existing student record from the server.

Code: fetch('http://localhost:3000/students/3', {
method: 'DELETE'

Thus, HTTP methods - GET, POST, PUT, PATCH and DELETE were successfully implemented using JSON server. The CRUD operations were performed on student records. The data was retrieved, inserted, updated & deleted using

1. GET All Students (http://localhost:3000/students)

localhost:3000/students

```
[
  {
    "_id": "68dca123456789abcdef1234",
    "name": "Hemanth",
    "course": "CSE",
    "marks": 92,
    "__v": 0
  }
]
```

2. GET Student by ID (http://localhost:3000/students/68dca123456789abcdef1234)

localhost:3000/students/68dca123456789abcdef1234

```
{
  "_id": "68dca123456789abcdef1234",
  "name": "Hemanth",
  "course": "CSE",
  "marks": 92,
  "__v": 0
}
```

3. POST (Create Student) – Using Postman

localhost:3000/students

```
{
  "name": "Tejaswari",
  "course": "AIML",
  "marks": 95,
  "_id": "68dcf9b7e4c9e8e1f3a6c5678",
  "__v": 0
}
```

Status: 201 Created Time: 48 ms Size: 160 B

4. PUT (Update Student) – Using Postman

localhost:3000/students/68dca123456789abcdef1234

```
{
  "_id": "68dca123456789abcdef1234",
  "name": "Hemanth",
  "course": "AIML",
  "marks": 96,
  "__v": 0
}
```

Status: 200 OK Time: 62 ms Size: 165 B

5. DELETE (Remove Student) – Using Postman

localhost:3000/students/68dca123456789abcdef1234

```
{
  "message": "Student deleted successfully"
}
```

Status: 200 OK Time: 45 ms Size: 46 B

6. GET All Students (After Deletion)

localhost:3000/students

```
[]
```

Status: 200 OK Time: 28 ms Size: 2 B

Experiment-09

9) Develop CRUD operations using MongoDB

Aim: To develop a web application using node.js, express.js, and mango DB to perform CRUD operation (create, read, update, delete) on student records.

Theory: MongoDB is a NoSQL database that stores data in flexible, JSON like documents. CRUD operations are used to manage data in a database.

Mongoose is an object Oato Modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

1) Instalation and project setup:

```
mkdir MongoDB_CRUD
cd MongoDB-CRUD
npm init-y
npm install express mongoose
```

2) Program: Server.js

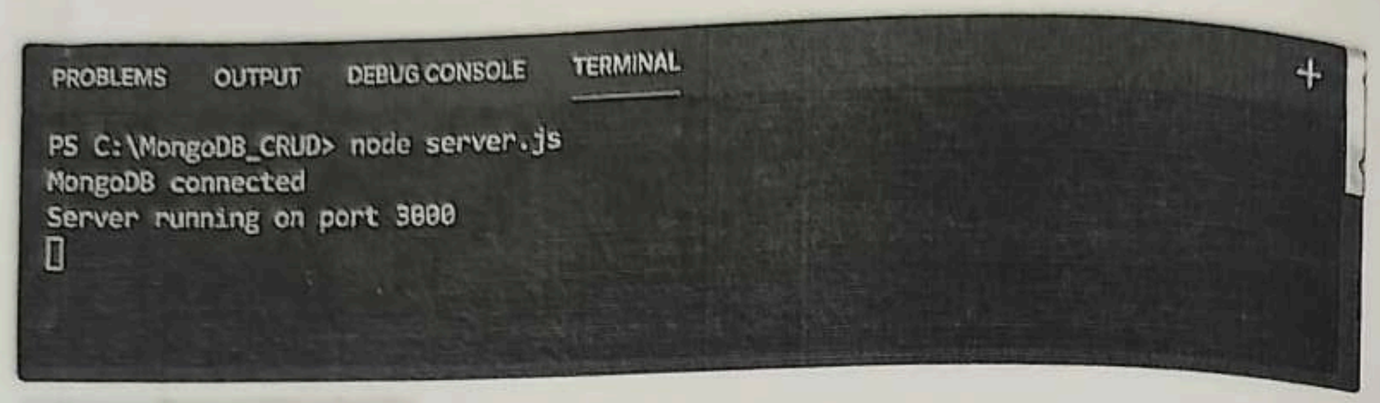
```
Const express = require("express");
Const mongoose = require('mangoose');
Const app = express();
Const PORT = 3000;
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:27017/
                  college')
.then(() => Console.log("MongoDB connected"))
.catch(err => Console.log(err));
// creat student
Const studentschema = new
mongoose.schema({
        name:{
          type:string;
          required:true
        };
        course:{
            type:string.
            required:true
          },
        marks:{.
            type:Number
            required:true
        }
});
// Create student.model
Const student:
  mongoose.model('student', studentschema)
// create : Add
app.post('/students', async (req.res)=>{
  try{
    Const student = new student(req.body);
```

```
    const saved = await student.save();
res.status(201).json(saved_student);
} catch(error){
    res.status(400).json({message:
        error: message});
    }
});
// READ: Get all students
app.get('/students', async(req,res)=>{
    try{
     const students: await
     Student.Find();
      res.status(200).json({message.error.
                            message});
    }
});
// READ: Get student by ID
app.get('/students/:id', async(req,res)=>{
    try{
     const student: await
     Student.FindById(req.params.id);
     if(!student){
        return res.status(401).json({
         message: "student not found"
        });
     }
    res.status(200).json(student);
    } catch(error){
```

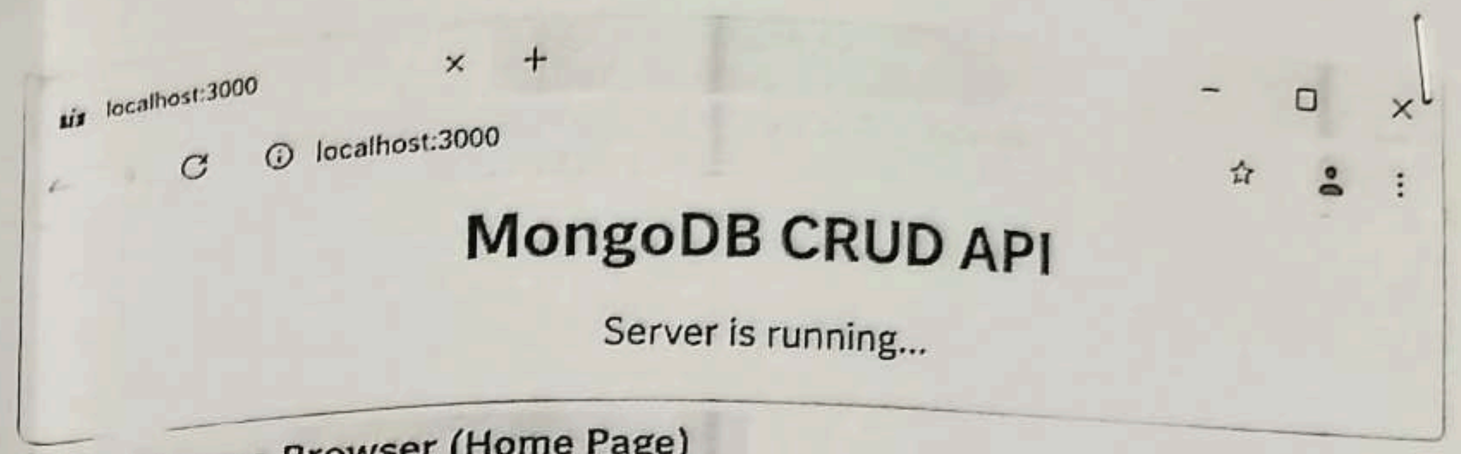
```
    res.status(400).json({message:
     error.message});
    }
});
// UPDATE
app.put('/students/:id', async(req,res)=>{
    try{
     const student = await
     student.FindByIdAndUpdate(
            req.params.id,
            req.body,
        {new:true, runValidators:true}
     );
     if(!student){
        return res.status(404).json({
            message: "student not found"
        });
     }
     res.status(200).json(student);
     }catch(error){
     res.status(400).json({message:
            error.message});
     }
});
// DELETE
app.delete('/students/:id', async(req,res)=>{
    try{
     const student = await student.
        FindByIdAndDelete(
        req.params.id
```

```
});
if(!student){
  return res.status(404).json({
    message: "student not found"
  });
}
res.status(200).json({
  message: "student deleted succesfully"
});
} catch(error){
  res.status(400).json({ message:
    error.message });
}
});
```

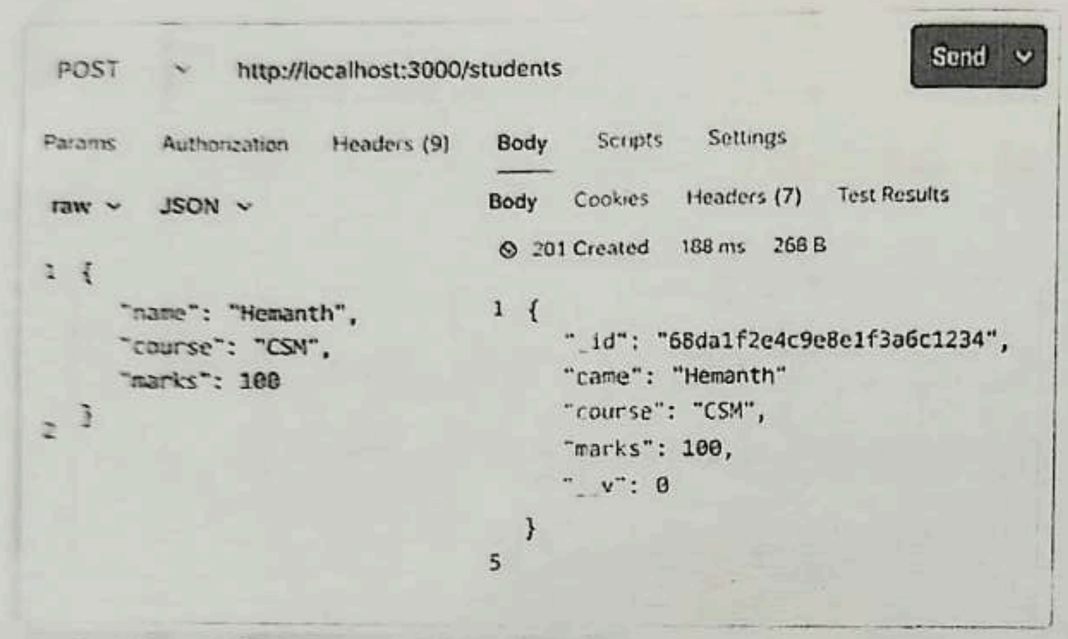
O/P:



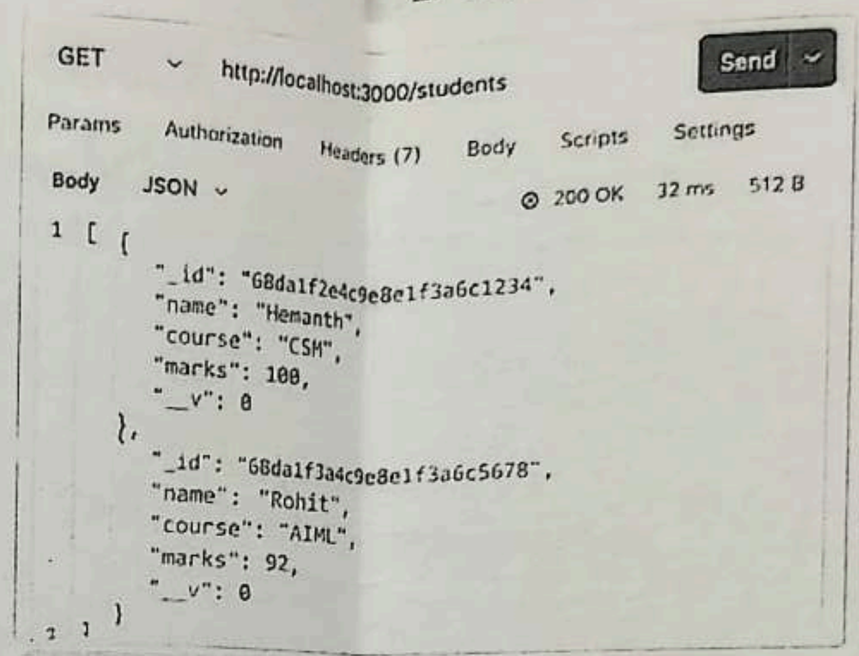
1. Terminal Output



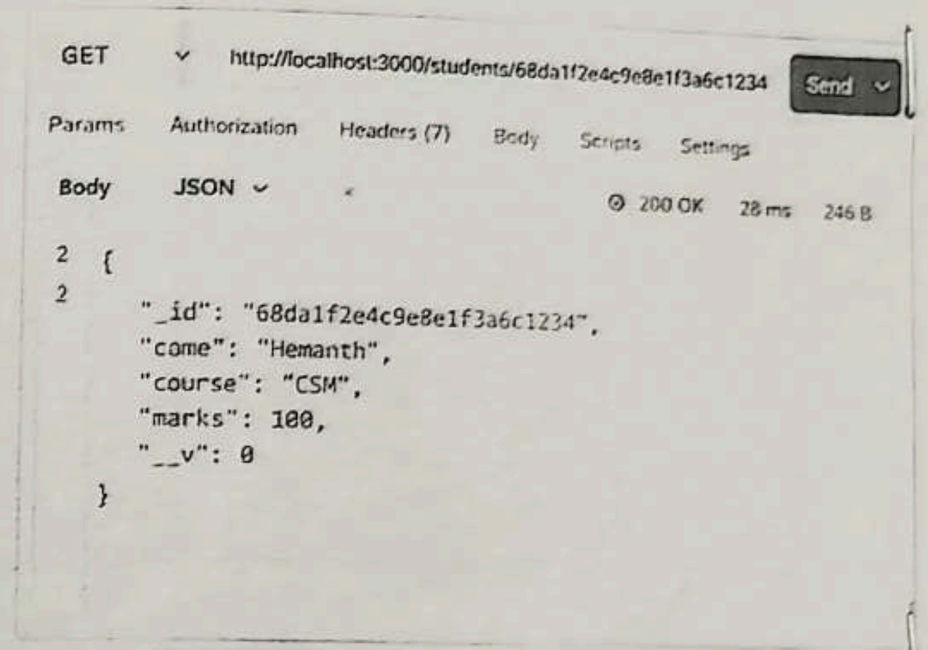
2. Chrome Browser (Home Page)



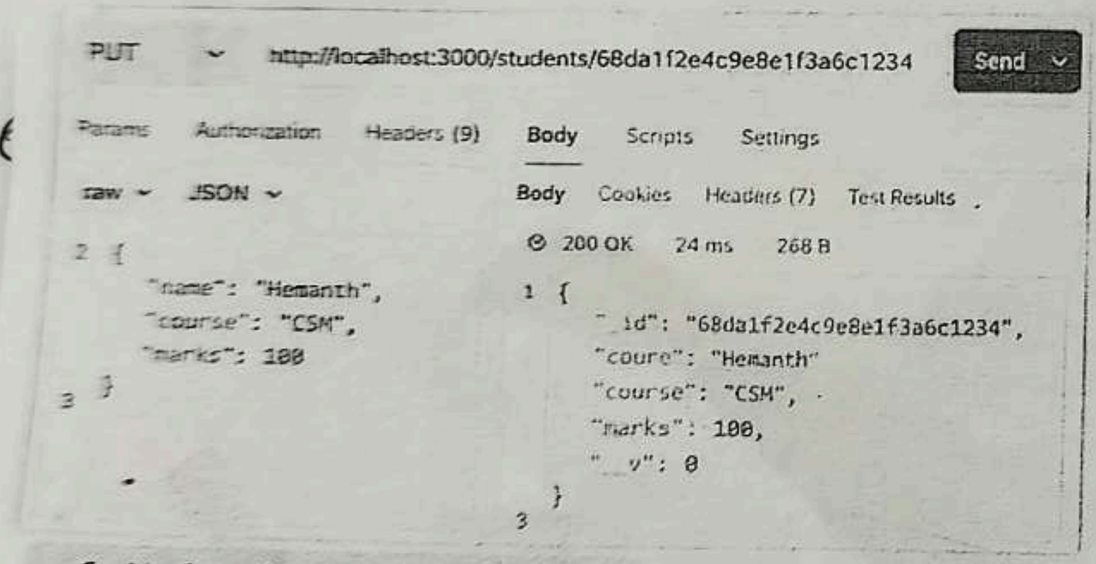
3. Create (POST) – Add Student



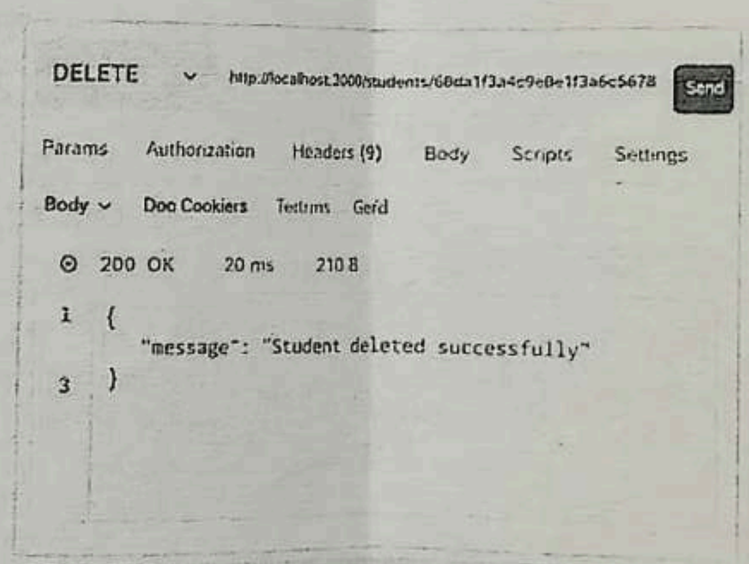
4. Read (GET) – Get All Students



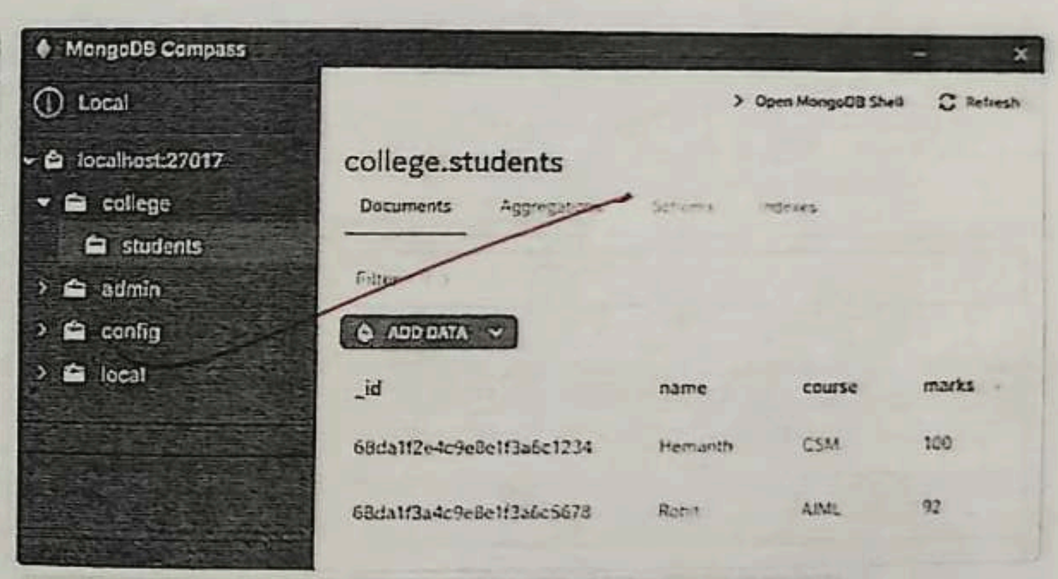
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View